

Phase 1 — What We Built

The goal of Phase 1 was simple: build a working trading engine from scratch, get it correct, and prove it works. No optimization, no shortcuts — correctness first.

The Core Engine

We started with the Event — a fixed 40-byte struct that carries every piece of information through the system (order ID, price, quantity, type, timestamp). Every component communicates exclusively through this struct. It's memcpy-safe, has no dynamic memory, and we packed the order side (Bid/Ask) into an unused bit of the type field so it stays exactly 40 bytes.

Then we built the Order Book — an array-based structure that tracks all resting buy and sell orders across a bounded price range. It supports inserting, cancelling, modifying, and partially filling orders.

On top of that, the Matching Engine — it takes an incoming aggressive order, walks the book at the best opposite price, and generates fill events using price-time priority. We also implemented Self-Trade Prevention (STP): if a trader accidentally sends an order that would match against their own resting order, the aggressor is cancelled instead of executing. We encoded participant identity into the upper bits of the order ID so no extra fields were needed.

The Strategy Engine sits between the replay and the matcher. It's a pluggable filter — in Phase 1 we built three strategies: pass-through (forwards everything), null (drops everything), and event-type filter (forwards only specific event types). It gives us a hook for future trading logic.

Replay & Persistence

The Replay Engine reads events from a CSV file and feeds them into the pipeline deterministically. The same input always produces the same output — this is the foundation for debugging and testing. We also built an in-memory version for unit tests. Both implement the same abstract interface so the rest of the engine doesn't care which one is running.

For persistence, we built an async PostgreSQL writer. Trade executions are pushed into a queue from the hot path and a background thread drains them to the database. The engine is never blocked by database I/O. Each replay run gets a UUID so results across runs can be compared.

Dashboard

We built a WebSocket dashboard in C++ using uWebSockets, which pushes a JSON snapshot every 200ms to any connected browser. A React frontend displays four views: live order book

depth, recent trades, replay progress, and engine metrics (throughput, latency, signals vs noops). The engine thread never touches the dashboard directly — a `SnapshotPublisher` sits in between with a mutex, so the dashboard cannot slow down the hot path.

Testing

112 tests covering every component. Key integration tests verify that the same CSV input produces identical executions on every run (determinism), that STP fires correctly end-to-end, and that out-of-order timestamps are detected. The PostgreSQL test is skipped when Docker isn't running.

Phase 2 — What We Built

Phase 1 proved correctness. Phase 2's goal was: make the engine deterministic in memory — no surprise heap allocations during a replay run, and all infrastructure needed for Phase 3 performance work.

Pool-Based Memory

We built `ObjectPool<T>` — a fixed-capacity pool where all storage is allocated once at startup. The `allocate()` call pops a pointer off a free-list stack in $O(1)$. `release()` pushes it back. No `malloc`, no `new`, ever during a replay.

The `StartupAllocator` creates four pools before the engine starts: one for Events, one for Orders, one for ExecutionReports, and one for strategy signals. After that point, no further heap allocation is permitted.

To enforce this, we built the `AllocationGuard` — it overrides all four variants of operator `new`. When the engine is running, any unexpected heap allocation increments a counter and can trigger an assertion. We wrote a full integration test that runs a replay inside the guard and asserts the count is exactly zero.

Making the Existing Engine Allocation-Free

This required refactoring three places in Phase 1 code that silently allocated during matching:

Order lookup (`std::unordered_map`) — each inserted order allocated a heap node. We replaced it with `FlatOrderMap`, a custom open-addressing hash table whose storage is a `std::vector` pre-sized at construction time. After startup, inserts and erases never allocate.

Price level order lists (`std::vector<uint64_t>` per level) — the first order at any price level triggered a vector allocation. We replaced it with `std::array<uint64_t, 16>` + a count field directly inside the `PriceLevel` struct. No per-level allocation.

Match output (`std::vector<Event>`) — each fill pushed into a vector. We replaced it with `std::array<Event, 16>` + an `event_count` field. Added an `event_span()` accessor so all existing range-for loops still work without change.

Cache Alignment

The `Event` struct grew from 40B to 64B (one full cache line) with `alignas(64)`. The 24 extra bytes got two new fields: a global sequence number and a reserved slot for Phase 3. We added two new cache-line-aligned pool types: `Order` (64B) and `ExecutionReport` (64B), plus `SignalRecord` (32B — two fit per cache line).

The SPSC queue got `alignas(64)` on its head, tail, and buffer — this prevents false sharing where the producer writing tail and the consumer writing head would otherwise fight over the same cache line.

Infrastructure for Phase 3

We built four new components:

`StartupValidator` — runs pre-flight checks before the engine starts: are pools properly allocated, is the clock monotonic, what NUMA nodes are available? On WSL2 it skips the NUMA check automatically.

`ThreadAffinityManager` — pins engine threads to specific CPU cores via `pthread_setaffinity_np`. On bare-metal Linux, a failure aborts startup. On WSL2, it just logs a warning and continues.

`TimestampProvider` — an abstract nanosecond clock with two implementations: `ChronoTimestampProvider` (portable, always works) and `RdtscpTimestampProvider` (reads the CPU timestamp counter directly, calibrated once at startup using a 100ms sleep, with automatic fallback to chrono if the CPU doesn't support RDTSCP).

`SymbolRouter` — routes events to the correct per-symbol pipeline. Each symbol has its own order book, strategy engine, and matching engine. Events for symbol 1 can never affect symbol 2's book. Supports up to 256 symbols with $O(1)$ lookup.

Dashboard Update

Added a fifth Pools tab to the React frontend showing: startup report (whether the allocator started, mlockall result, validation pass/fail), and a live pool utilisation table for all four pools — capacity, used slots, available slots, and exhaustion count. Exhaustion count highlights in red when non-zero.

Result

187 tests passing. Zero unexpected heap allocations during a full replay run. The engine is now ready for Phase 3: prefetching, NUMA tuning, branch minimization, and latency benchmarking.